读懂 AI Infra，看这六个关键词就够了

原创 腾讯程序员 腾讯技术工程 2026年1月23日 01:14 广东 197人



> 2025年初，DeepSeek的走红让更多人明白，不仅仅是模型本身，训练和推理过程中工程上的优化同样重要。元旦假期看了朱亦博老师的一篇帖子，很受启发，2025年过去了，我想应该尝试来把亦博老师总结的25年AI Infra六个重点方向尽可能以一些简单易懂的方式向大家介绍一下，也希望让更多的同学可以对这一年里AI Infra的发展有一些简要的了解。

作者：dorianbian，larrylyyu

一、分布式推理

前置知识点的一句话理解：

1. 混合专家模型 MoE：由门控网络+若干个专家模型组成，门控网络负责选择适合某类问题的专家模型进行激活并处理请求，不同的专家模型擅长处理不同领域的问题，单一请求一般只激活少部分专家进行处理，即稀疏激活特性。

2. 自回归仅解码架构 Decode-Only：基于前文逐步生成下一个词的方式，推理分为prefill和decode两个阶段，prefill将输入计算为K、V，decode基于前序得到的K、V逐词生成输出，意味着P和D两个阶段对资源的需求完全不同。

目前主流的MoE模型 + Decoder-Only推理架构，针对MoE的稀疏激活、推理不同阶段资源需求不同的特性，进一步将不同阶段或模块解耦进行部署以提升效率，主要为以下三个方向：

1.PD分离的持续演进

Decode-Only最初是Prefill和Decode两个阶段混合部署在一起的，在优化TTFT（首个token响应时间）和TPOT（后续平均每个token生成时间）两个核心指标的过程中，出现了两个棘手的核心问题：Prefill-Decode两个阶段的强干扰和资源分配与并行策略的耦合。

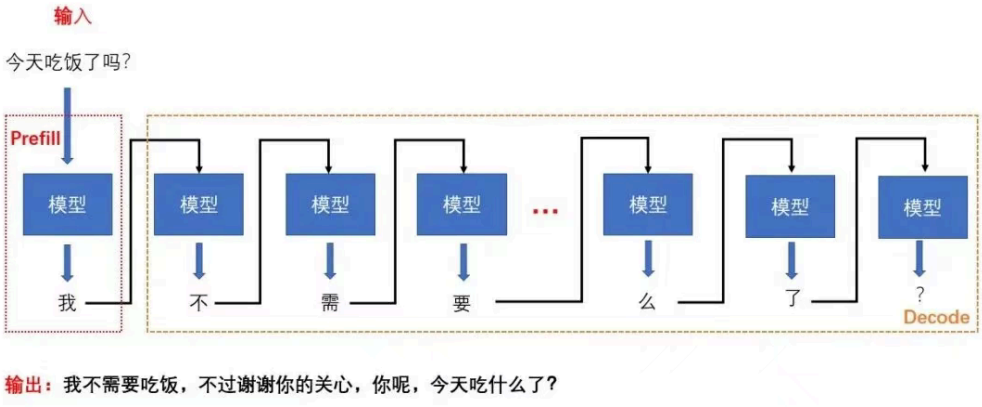
（1）PD分离部署的提出

为了通过解决Prefill和Decode两个阶段资源需求不同而带来的强干扰和并行策略耦合的问题，来提升TTFT和TPOT两个核心指标，DistServe论文首次提出了PD分离的概念。

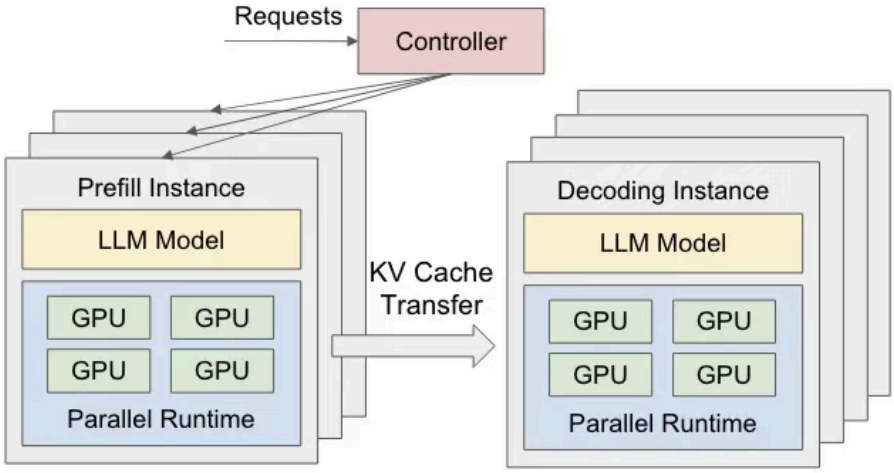
Prefill阶段：该阶段是将输入的prompt转换为Q、K和V并生成首个token，同时把K、V存储用于Decode阶段生成输出使用，因此预填充阶段通常是计算密集型的，核心优化目标是TTFT。

Decode阶段：该阶段是使用输入的prompt和生成的首个token的K、V值，来生成接下来的一个token1，再使用输入prompt+首token+token1来生成token2，依次迭代直到完成全部输出，因此解码阶段通常是访存密集型的，核心优化目标是TPOT。

以一张图来直观说明两个阶段的关系（同时也可以思考下，P和D两个过程中，为什么只存K、V cache，而不存Q cache？）：



Prefill 和Decode阶段间KV cache的传输：在具备IB（或Roce）网络的环境下，把P、D两个阶段分别部署在不同的设备上，或在一个设备内把P、D两个阶段分别部署在不同的GPU上，这样就规避了前文中提到的不同阶段强干扰和并行策略耦合的问题，具备了独立优化TTFT和TPOT的可行性。下图为P、D分离部署的逻辑架构图：



根据论文实验数据和一些具有大规模用户的推理服务部署实践情况来看，同样的资源，PD分离模式部署确实可以在相同TTFT、TPOT要求下承载更多的推理请求，或者为相同的推理请求提供更优的TTFT、TPOT体验。

（2）PD分离部署的进一步优化

PD分离的提出，解决了预填充和解码两个阶段优化过程中相互强干扰和强耦合的问题，大幅优化了推理服务过程中的吞吐量和时延等指标，但在工程实践中，特别是混合长度请求的场景下，存在KV的生产者-消费者失衡的问题：预填充实例处理请求的速度和解码实例消费的速度不匹

配，导致部分实例资源利用过低或过载的问题。为了应对这样的问题，DOPD论文提出了动态调整P、D实例的策略，从而进一步提升资源利用率和推理服务效率。

注意到：推理过程中P、D实例负载很大情况下取决于推理请求的长度，而不同推理请求的长度存在较大差异。因此动态PD分离应该具备以下能力，来解决复杂推理请求场景下P、D失衡问题：

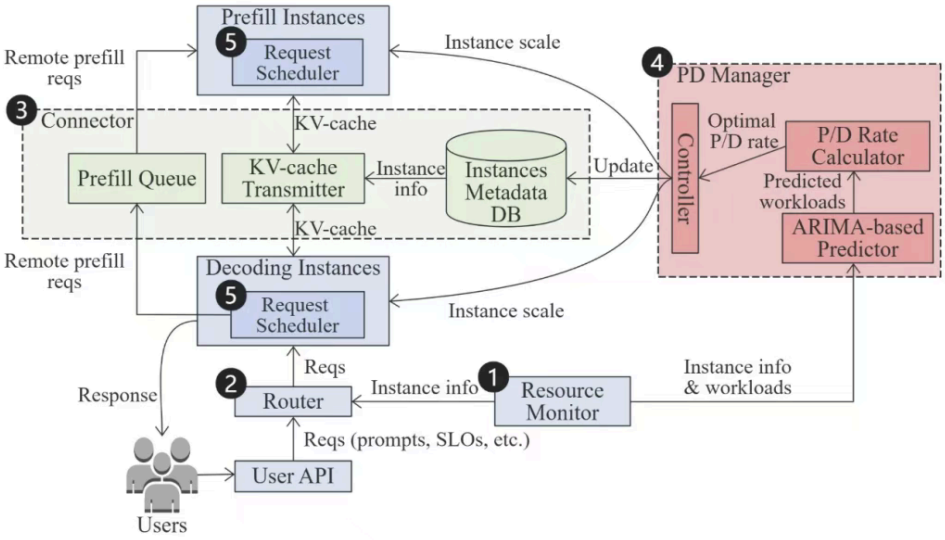
1. 具备推理请求短期负载的预测能力

2. 结合短期预测和当前实例负载计算出建议的P、D实例配比，并且该阶段的时间&资源开销是极小的

3. 缓解高并发场景下混合长度请求给预填充 - 解码解耦架构带来的干扰问题

4. 基于建议配比，实现P、D实例的自动扩缩容

DOPD论文中，设计了一种动态最优PD分离调节系统，主要包括五个组件：资源监控器、请求路由器、KV通信连接器、P&D实例管理器和请求调度器。该系统架构图如下：



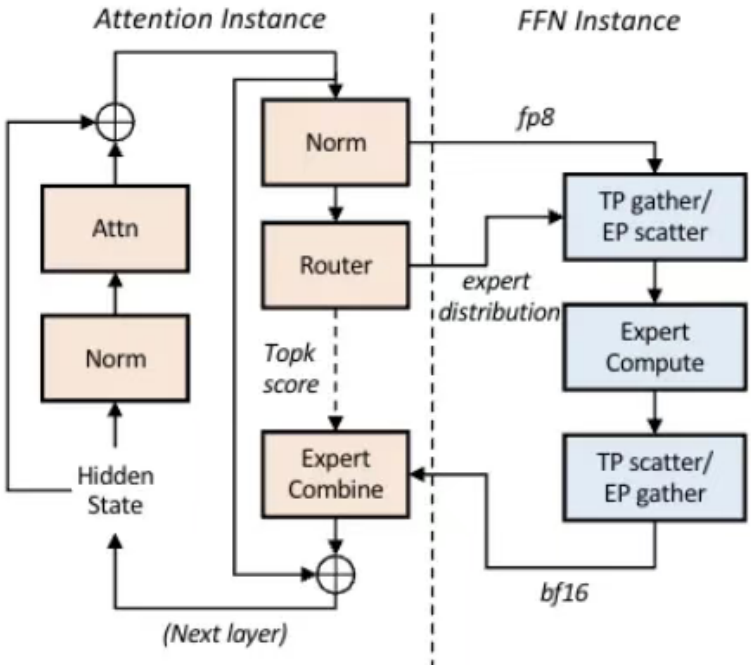
工作原理简述为，资源管理器负责监测P、D实例集群的各项运行数据，并将这部分数据提供给请求调度器、P&D实例管理器和请求路由器。P、D集群各自实例的数量，和其张量并行度（TP），则由P&D实例管理器来实时调整，确保整体资源利用率处于合理水位。当一个用户请求进来后，请求路由器将结合各D实例上的KV命中率和负载情况选择该请求的目的D实例，而需要远端P实例进行预填充时，则依赖KV通信连接器与远端P实例进行通信。当高并发且混合长度请求时，请求调度器会将请求分组进行批处理，以提升系统整体的吞吐或时延效率。

根据论文实验数据来看，动态PD分离的实践除进一步优化推理服务整体的吞吐和时延外，尤其适合负载波动大、混合长度请求的场景。

2.AFD的提出

由于预填充和解码两个阶段对资源需求显著不同，前文已经介绍了PD分离相关内容。那么是否可以将单一阶段展开分析，来尝试进一步优化。注意到基于Attention的LLM模型通常由注意力模块和前馈网络模块组成，解码阶段中，注意力模块的参数量少但需要大量KV交互，是访存密集型的，而前馈网络模块参数量大，是计算密集型的，因此Step-3模型论文提出了AFD（Attention -

FFN Disaggregation）架构，即将注意力模块和前馈网络模块部署在不同的设备上，在PD分离的基础上进一步优化资源的利用率和推理服务效率。下图为AFD的架构图：



论文中实验解码环节采用H20部署Attention实例，H800部署FFN实例，数据显示AFD部署可以有效提升推理服务的效率和降低整体的硬件部署成本，同时也提出FFN实例部分也具备使用国产化硬件的可行性，打破对高端GPU的单一依赖。

3.跨机EP的持续优化

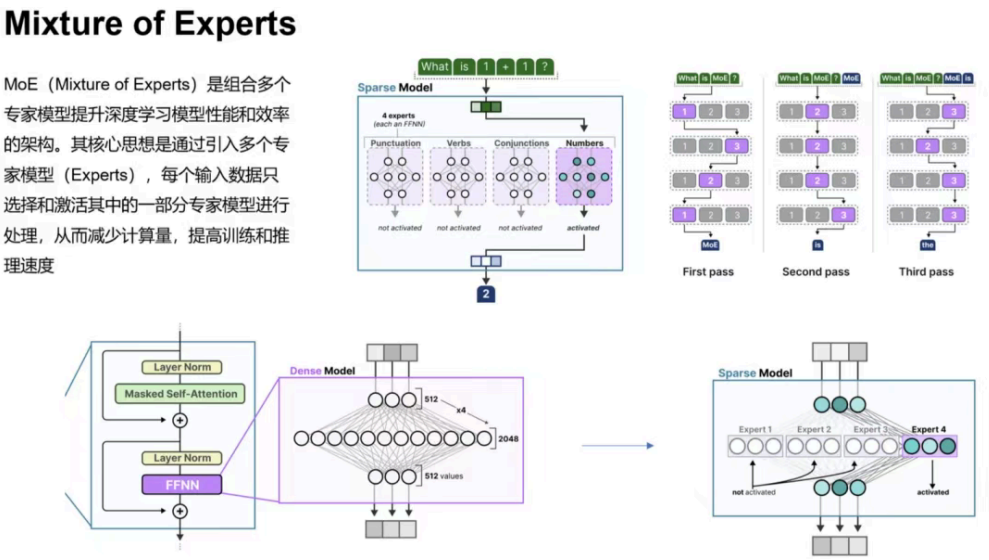
我们已经了解MoE模型由若干专家模型组成，主流的大模型拥有几千甚至更多的专家数量，例如DeepSeek V3系列有约1.4万个专家模型，如果所有的专家模型全部部署在一张GPU卡，或一台服务器上是远远无法承载的，主要原因为以下三个：

1. 单卡或设备显存无法承载过多的专家及其参数

2. MoE模型的稀疏特性，每次输入仅激活少量专家，其余未激活部分算力处于闲置状态

3. 千亿或更大的MoE模型对通信开销、扩展效率有更高的要求

由于MoE模型的稀疏激活特性，在多机部署时GPU之间的通信（all-to-all）仅需按需传输，仅把对应的token传给激活的专家模块。仅用NCCL在这种通信模型效率很低，导致带宽资源大量浪费，原因是NCCL更擅长全量而密集的通信。



（1）DeepEP

由DeepSeek开发的一个开源项目，是专门为混合专家模型（MoE）+专家并行（EP）设计的通信库，优点具体如下：

1. 专门针对稀疏激活专家特性设计的通信库，仅按需通信，带宽和时延大幅提升

2. 支持原生FP8精度的通信

3. 具备NVlink-RDMA非对称带宽域的转发能力

4. 通信-计算重叠

根据开源项目中提供的实测数据来看，在典型的DeepSeekV3/R1业务模型下，跨机带宽和时延表现可观。

（2）TRMT

DeepEP针对MoE模型稀疏通信的特点进行了专项的优化，使模型跨卡/机通信的效率大幅提升，但其实现和实验均是基于IB网络进行的，但在RoCE网络上表现却大打折扣。腾讯网络平台部门在DeepEP的基础上，结合TRMT技术进行针对性优化，大幅提升了DeepEP通信库在RoCE网络下的效率。目前这些成果已经开源在DeepEP社区，腾讯网络平台部门也得到了DeepEP社区官方的致谢。TRMT的简介可以参考《DeepSeek致谢腾讯大模型网络提速技术方案贡献》

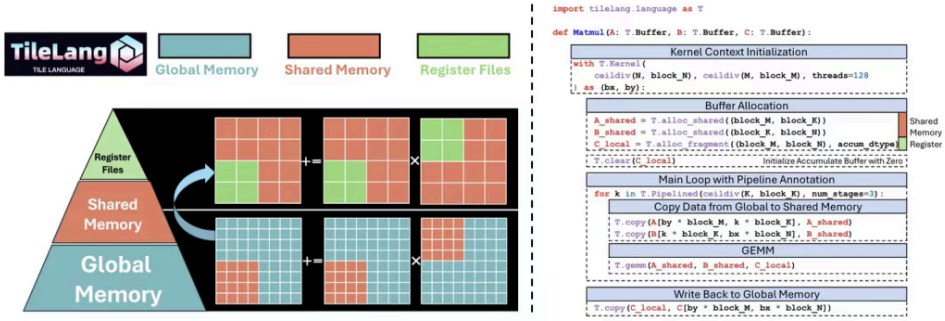
DeepSeek致谢腾讯大模型网络提速技术方案贡献

二、面向Tile的开发语言

高性能训练or推理的内核或算子优化往往是一件很精细化的工作，要精确定义thread、block、grid如何调度，或各种规格硬件的的tensor计算核心如何使用，以及各级缓存如何访问。传统的优化与加速方式都是针对硬件特性的精细化设计+复杂的调优手段来实现的，这些复杂的工作的核心目的还是如何更加高效的处理数据，来提升算子或模型整体的效率。那么是否可以有一种开发语言，让开发者仅聚焦于数据流的开发，而不用过多关注更加底层的精细化调度呢？

TileLang

这个开发语言由TileLang这篇同名论文提出，是一种将数据流与调度逻辑彻底解耦：让开发者只需专注于数据流逻辑，而调度策略交由编译器完成，达到以更简洁的代码表达复杂计算，并获得最优性能的效果。论文中给出了一个矩阵乘法的例子，仅需简洁的几种矩阵定义、内存分配、流水线定义和算子调用的高级语法，即实现了一个M*K矩阵与K*N矩阵的乘法运算，参考下图：



(a) Efficient GEMM with Multi-Level Tiling on GPUs　　(b) Describing Tiled GPU GEMM with TileLang

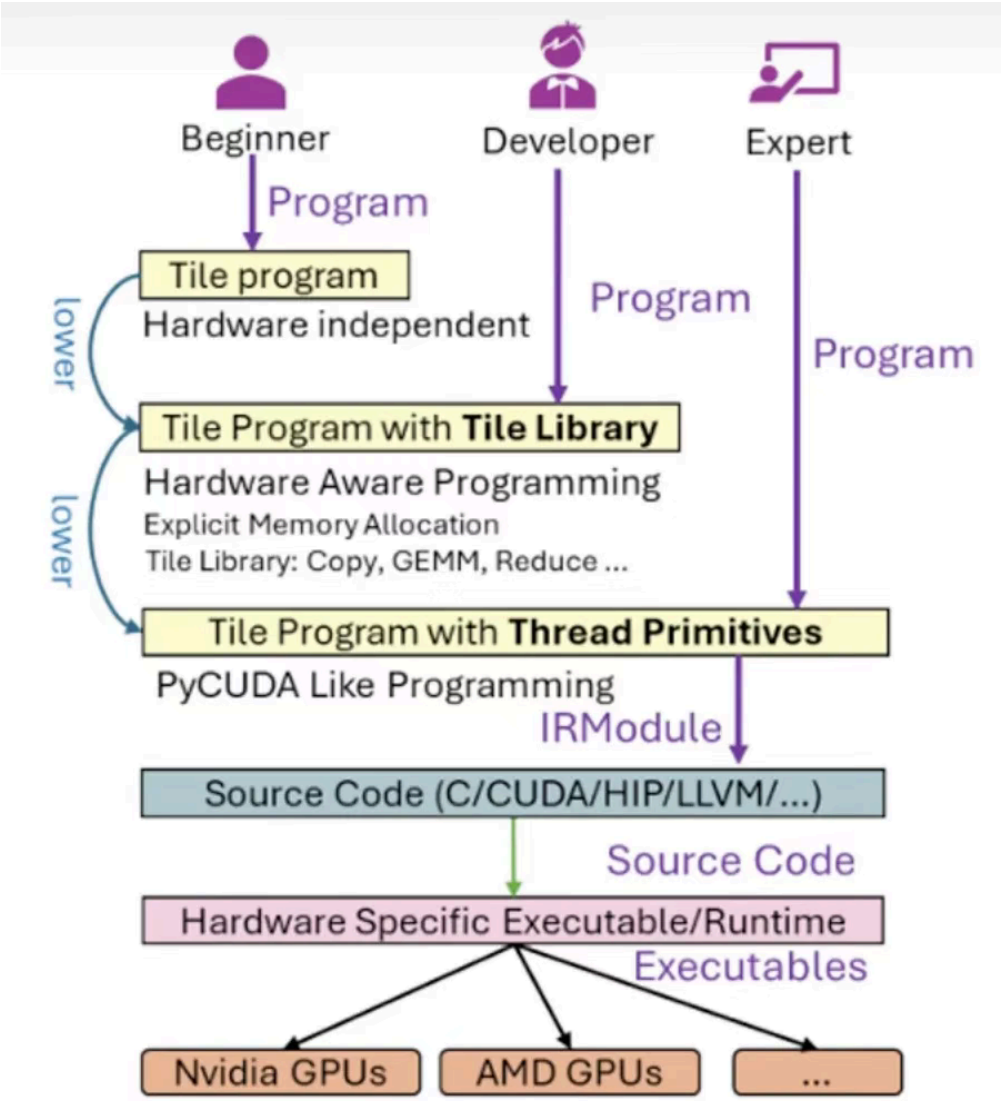
TileLang也提供了三个层级的编程方式，来满足不同层级的开发者：

入门版：仅描述数据块和计算逻辑，其余硬件实现细节和优化全部由编译器完成

进阶版：在入门级的基础上，除了描述数据库和计算逻辑，可以调用一些封装的算子，兼顾开发效率和灵活性

精细版：可以进行精细化的线程级开发，适用于极致性能优化场景

同时也支持几种层级混合使用。下图为TileLang三个层级的示意图：



传统的优化方式以手工适配为主，TileLang可以进行自动调度。根据论文中的实验数据来看，TileLang可以以少量代码的前提下，提供更优的模型效率。此外，DeepSeek V3.2官方文档中也明确说明，部分算子采用TileLang进行了重构。

三、RL训推分离

我们知道，强化学习训练使用的数据集是由模型推理产生，或模型推理产生+人工标注产生。如果把训练任务和生成数据集的推理任务部署在同一集群上，自然部署难度和调度难度都是比较低的，但会存在以下几个问题：

1. 训练和推理两个阶段的优化思路是完全不同的，两个阶段耦合在一起优化难度极大且会相互影响

2. 训练和推理对硬件的需求不同，无法通过异构硬件部署的形式提升性价比

3. 训练或推理各自过程中如出现异常，则会影响到对方环节

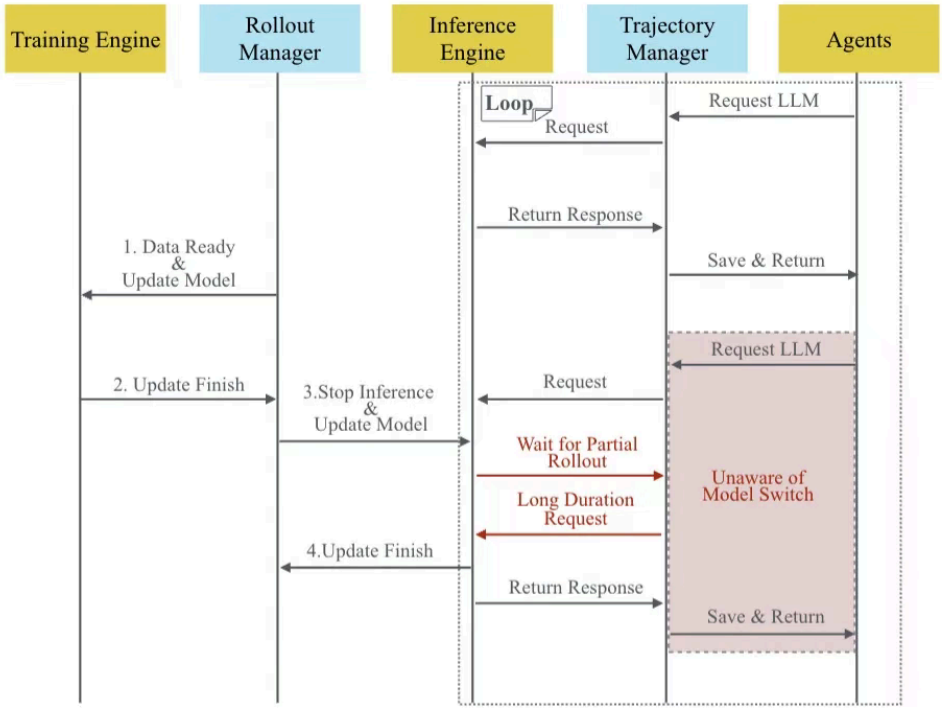
还是同样的思路，如果把两个阶段解耦开来独立优化，便比较容易的解决上述的几个问题。但是在训推分离的实践中，又会出现两个新的问题：

1. 推理的输出可能经过agent复杂的处理逻辑，比如和记忆有关

2. 训练和推理流程间的时序依赖关系，很容易造成“气泡”式的算力资源浪费

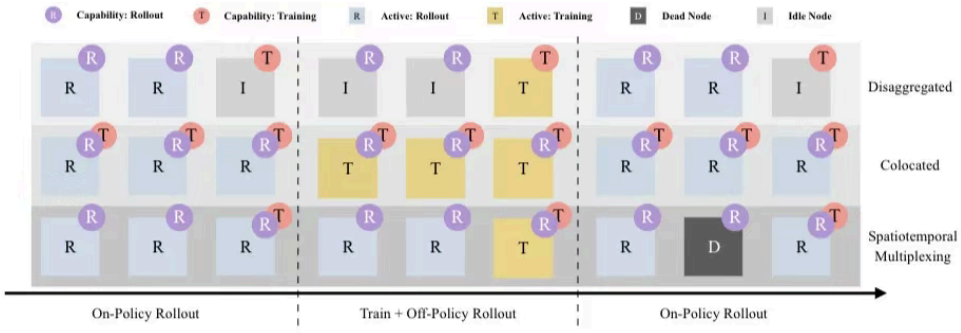
1.SeamlessFlow的训-推数据交互平面机制

针对训推分离架构的第一个问题，SeamlessFlow论文中设计了一个在推理生成阶段中，介于Agent和LLM之间的数据交互平面，其核心是一个“轨迹管理器”，即把agent调用llm的全过程，以及全部的输入输出全部精准记录，从而保障一致性，同时这个过程对Agent完全透明。数据交互平面的示意图如下：



2.SeamlessFlow的标签资源调度机制

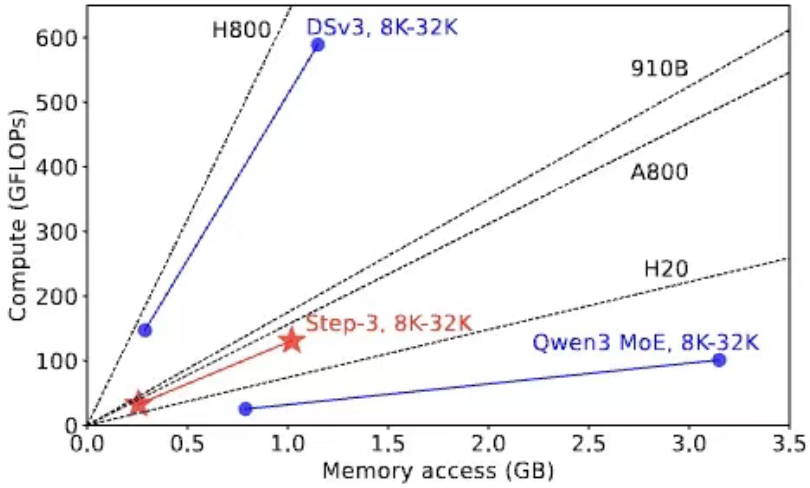
针对训推分离架构的第二个问题，恰好是训推一体架构的优势，SeamlessFlow论文中给出了一种比较好的解决方案：不在分离和一体中作抉择，而是选择灵活的资源调度方式。其机制简单来说是通过标签机制来给资源分配任务，即某些资源可以按需灵活执行推理或训练阶段的任务，从而实现计算资源气泡的消除。按标签资源调度机制示意图如下：



四、模型-系统的协同设计思路

包括前文中所涉及的，几乎所有的模型训练或推理优化思路都是基于模型的特点，来针对性的去优化。如果模型在设计的时候，本身就结合更高性价比的硬件去设计的话，那么在效率、性价比和优化难度上将有显著优势，Step-3模型就是在模型-系统协同设计的思路下设计完成的。

Step-3论文中提出了一个“算术强度”的概念，即和显存交互平均每个字节所需要运算的次数，“算术强度”和ATTN模块的设计有关，和batch size、上下文长度这些参数都无关，同时，GPU硬件的算力和显存带宽也是固定的，因此硬件的算力-带宽比（roofline）也是固定的。因此，当模型的“算术强度”高于硬件的“roofline”时，那么硬件的算力成为瓶颈，反之则内存带宽成为瓶颈。在设计的模型的ATTN模块时，”算术强度“应尽可能接近主流硬件的“roofline”，则整体效率较优。如下图所示，step3的设计，“算术强度”是接近A800或昇腾910B的。



量化的设计：针对于ATTN模块，如果采用MTP或者KV cache与计算量化精度不同的话，模型的“算术强度”会发生变化，依然如上图所示，Step-3模型无论“算术强度”是提升还是降低，均有可以匹配的硬件可用，因此量化方案的灵活性在模型-系统协同设计时也是需要考虑的一个因素。

MoE模型与系统架构的联合设计：针对FFN模块，主要以算力需求为主，参考论文中的公式：

$$\text{BatchSize_MoE} >= \text{FLOPs} / (2 * S * \text{Bandwidth})$$

可以清晰理解，针对特定的卡型如想提升计算效率，要么提升BatchSize，要么让MoE模型的激活更加稀疏。然而这里又会产生一个新的权衡点：在AFD架构下，如果BatchSize过大，网络传输耗时增大从而增大TOPT，因此我们在做模型设计时，需要综合设计模型和系统，参考论文中的公式：

$$S >= (H * \text{FLOPs} * L) / (\text{Net} * \text{Bandwidth} * \beta)$$

β是一个和FFN的量化精度、AFD流水线级数和目标TPOT几个确定数据有关的常数。将原文中的公式稍作整理：

$$S/(H * L) >= \text{FLOPs}/ (\text{Net} * \text{Bandwidth} * \beta)$$

可以很清晰的发现，如想达到比较高的效率，需要模型的稀疏度S、隐状态值H、层数L（公式左侧）与卡型、网络带宽和期望TPOT值（公式右侧）进行合理的设计与匹配。

五、Agent Infra

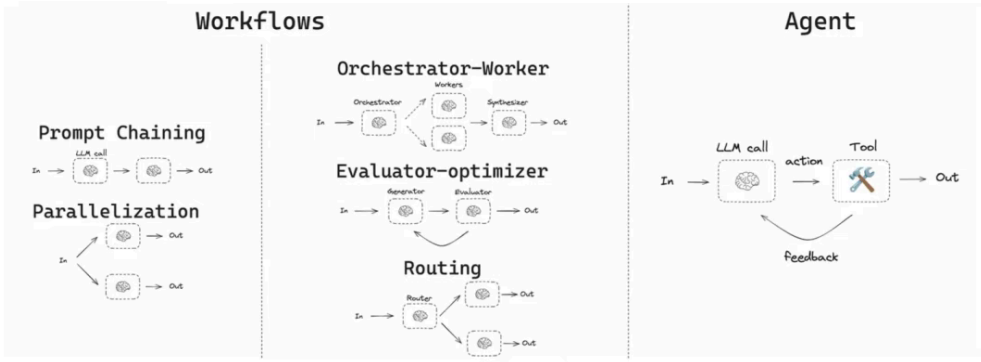
关于Agent Infra内容相关的内容，腾讯内部的知识社区有几篇文章很不错，这里为大家简要介绍一下几篇文章的关键信息：

1.《一文讲懂Agent及主流Agent框架介绍》

主流agent框架及对比分析（github上热度top5）：

Agent框架	适合场景	优势	不足
AutoGPT	各类通用任务，完全发挥自主性	1. 完全自主执行 2. 任务分解与多步执行 3. 记忆和持续学习	1. 复杂任务场景前后文一致性问题 2. 高成本和效率问题 3. 操作可控性较低
LangGraph	可明确拆解任务步骤	1. 灵活的多步骤控制 2. 原生支持短长期记忆 3. 易调试和全链路可观测	1. 自主性有限 2. Agent模式不成熟
Dify	可明确拆解任务步骤	1. 低代码，易用性与低门槛 2. 强大的模型与工具能力	1. 功能广而不精 2. 需在简单和复杂场景之间找到平衡
CrewAI	任务步骤不固定，需让Agent自己探索	1. 工具和生态集成 2. 灵活性与深度定制	1. 特定功能支持有限（如代码沙盒）
AutoGen		1. 原生多代理支持 2. 灵活的对话流程控制 3. 可观察调试支持	1. 社区生态尚处于起步阶段

Workflow vs Agent



Workflow：适合任务步骤确定 + 条件有限的流程，通过预设路径执行任务

Agent：适合任务步骤无固定分支，需要在对话上下文里做决策、需要跨工具动态组合、需要“问一句 → 查一下 → 再决定”

2.《MCP技术浪潮中的Agent应用开发新范式》

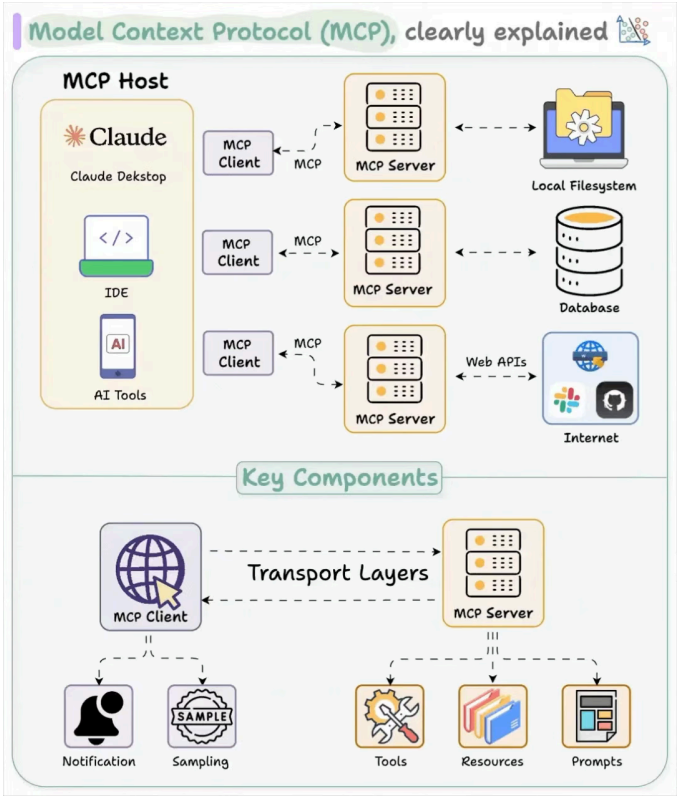
MCP是一个标准协议，定义了AI应用与外部工具和数据源的交互标准，MCP 的目标是创建一个通用标准，使 AI 应用程序的开发和集成变得更加简单和统一。实际在使用的过程中，可以通过对话的方式告诉agent你想要做什么，模型就会通过MCP协议传递相应的数据和操作给到MCP应用，执行对应任务。

MCP的组件包括：

MCP Server：提供工具执行、资源访问等功能，与MCP Clint交互

MCP Client：作为AI模型与Server的桥梁，处理请求和响应

MCP Host：发起请求的应用程序（如Claude Desktop或IDE），运行在用户设备上



MCP vs FunctionCalling：FunctionCalling的描述各厂商存在差异，模型迁移成本高；同时也存在效果不稳定、容错性低等问题。而MCP采用标准协议与接口，生态更好

3.《Agent的技能“陷阱”：Skills安全性深度剖析》

Skills与MCP是构建Agent的两个重要技术标准，Skills管“教方法”，MCP管“用什么做”，对比如下：

对比维度	Agent Skills	MCP
定义	封装专业知识、SOP流程和判断逻辑的模块，告诉Agent怎么做(How-to)。	标准化连接协议，赋予Agent操作真实世界数据的能力，告诉Agent用什么做(With-what)。
架构与性能	● 渐进式披露架构（Index -> Retrieve -> Load） ● 低延迟，首次激活100-200ms，后续无额外延迟 ● 低Token消耗 - 渐进式加载，元数据约100 tokens，仅在需要时才将完整指令注入上下文 ● 本地执行，支持纯离线	● C/S协议架构（Connect -> List All -> Choose） ● 较高延迟，远程连接时每次调用有网络RTT ● 中高Token消耗 - 连接即加载，工具数量越多其描述占用的token越多，多轮对话中的工具输出容易挤爆上下文窗口 ● 多种连接方式，支持Stdio/HTTP/SSE，部分依赖网络连接

部署与维护	● 极简部署 (Copy-Paste)，可热插拔，无状态/按需加载，运行于Agent沙盒中 ● 简单版本控制 - 通过git管理，更新简单（git pull） ● 易于共享 - git仓库或文件夹打包，社区生态丰富 ● 易于修改文本和Prompt，非技术人员也可维护业务逻辑	● 运行于用户配置的nodejs/python等运行环境，需要在Agent中手动添加JSON配置连接信息 ● 复杂版本控制，需重启服务器、同步配置、可能需要数据迁移 ● 复杂共享，需要分享代码、配置、部署文档，用户需自行部署 ● 需要懂技术的人员进行开发与维护业务逻辑
安全特性	● 通常运行在Agent沙盒中，可执行任意代码，依赖Agent厂商的沙盒安全性 ● 可通过A.I.G等专业安全扫描工具进行源码审计（传统代码审计工具无法检测Skill.md中自然语言层面的注入风险） ● 所有Skills同等权限，无细粒度控制 ● 主要风险包含语义劫持与幽灵指令（提示注入）、恶意后门脚本	● 设计初衷就是访问外部资源(工具/数据)，可部署于Agent本地环境与远程容器环境中，依赖基础设施安全性与用户的安全意识 ● 远程MCP无源码，本地MCP有源码，均需通过A.I.G专业等进行扫描 ● 支持工具级访问控制、OAuth认证机制 ● 主要风险包含工具投毒与覆盖（提示注入）、恶意行为、服务器漏洞
适用场景	● 重复性任务和流程 ● 专业知识和最佳实践封装 ● 文档模板和品牌一致性 ● 团队协作和知识共享 ● 快速部署和低维护场景	● 实时数据库查询 ● 外部API和服务集成 ● 知识截止日期后的信息 ● 服务器端复杂计算 ● 企业内部系统能力连接

Skills的信任陷阱：

本地静态文件、低延迟、低 Token 消耗、易部署与共享，但运行于 Agent 沙盒，权限控制粗糙，使得恶意攻击异常简单。

1、 按需激活机制的风险：

a) 语义劫持：攻击者可以精心构造一个恶意的Skill描述，当用户发出模糊指令时，Agent可能会错误地激活恶意Skill而非预期工具。

b) 幽灵指令：攻击者可以利用Agent加载SKILL时的技能详情文档，嵌入隐蔽和看似无害、实则恶意的幽灵指令。

2、 隐藏在Scripts目录中的恶意脚本：SKILL.md中的指令通常会引导智能体调用本地的辅助脚本来完成复杂任务。由于Agent调用的Bash通常拥有当前Agent用户的Shell权限，这意味着脚本可以读取SSH密钥、修改系统配置、安装后门，甚至发起反向Shell连接。

3、 不安全配置导致的隐密后门：Skills支持部分特殊配置，如声明allowed-tools: Bash 则意味着当该SKILL进入激活状态后（单次同意），后续同一对话中执行任意Bash命令都不再需要用户进行二次确认。朱雀研究发现这可能被攻击者滥用为实现零点击RCE与持续控制的捷径。

Skills安全建议：

- Skills开发者：源头管控
- Agent厂商：平台治理
- Agent用户：意识防线

4.《Agent 时代，客户需要什么样的infra？》

当前成熟的Docker 容器、传统虚拟机（VM）以及 Serverless (FaaS) 都不适合承载Agent业务，原因如下：

容器技术的问题：安全性，原因是Docker 容器本质上是进程级隔离，所有容器共享宿主机的 Linux 内核， Agent 的核心逻辑是 LLM 生成一段不可信、不可预测的代码并执行它。在多租户环境下，这意味着一个用户的恶意代码可能搞垮整个集群或窃取其他用户数据

传统虚拟机的问题：启动时延，Agent 的思维链是“思考-写代码-运行-看结果-修正”。这是一种高频交互。用户无法忍受每次让 AI 执行任务都要分钟级的等待，Agent需要毫秒级的环境准备速度

Serverless 的问题：状态保持（Statefulness），Agent 的任务通常是连续的。第一轮加载了100MB 的CSV文件到内存，第二轮分析数据时，这100MB 数据必须还在。如果采用serverless方式部署，那么为了维持上下文，FaaS 需要频繁地从外部存储（S3/Redis）加载和卸载数据，导致巨大的 I/O 开销和延迟

Agent Infra 落地关键技术要求：

- 强隔离与安全性：Agent生成的代码不可信，必须防止逃逸。
- 长程任务稳定性：任务可能运行几分钟甚至几小时（如深度调研、代码重构），连接断开不能导致任务失败。
- 状态管理与持久化：运行过程中的文件、内存变量在Crash后能否恢复？
- 可观测性与审计：必须知道Agent在沙箱里到底干了什么（执行了什么命令，访问了什么URL）。

腾讯云的最佳实践：

极致性能：

- Cube沙箱技术，实现80ms服务交付
- MVM快照+镜像预热技术消除冷启动

全链路安全：

- 独立Guest Kernel硬隔离
- 元宝的最佳实践：业务逻辑与执行环境物理分离

智能成本控制：

- Serverless架构，不请求无资源消耗
- 暂停/恢复机制实现零闲置成本

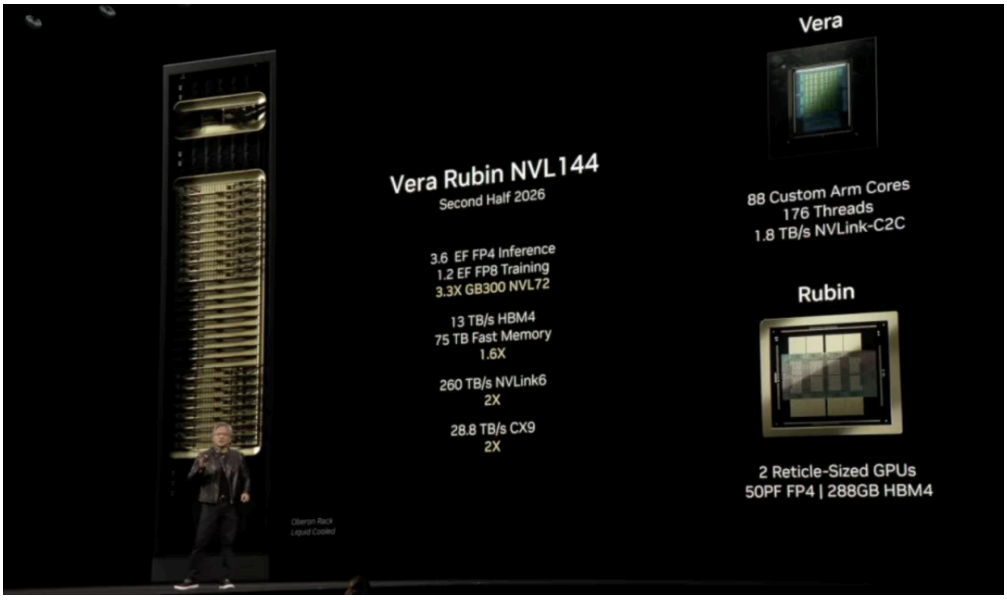
企业级工程化：

- VPC内网打通+共享文件系统多沙箱共享数据
- 兼容E2B协议无缝迁移

六、超节点形态的硬件基础设施

训练、推理的跨卡/跨机通信目前已经成为刚需，相较于RDMA，GPU之间的专有通信通道更为高效。超节点核心就是让更多的卡可以通过专有通信通道进行互通，形成一个高效的超大GPU域。

Vera Rubin NVL144：单域包括144颗Rubin GPU，支持FP4，单域带宽高达260TBps



超长上下文模型的推理

在推理场景中，模型的上下文长度的增长对算力的需求是平方级增长，对显存的需求是线性增长，结合上文中提到的Prefill-Ddecod分离、Attn-FFN等分离部署的思路，对卡间通信带宽的需求也是极大的，因此更大规模的NVlink高带宽域将为超长上下文模型的推理服务的提供更加有利的基础设施环境。NVIDIA官方的技术博客中也有提到，面对未来1M上下文长度的模型即将成为主流的趋势，采用Rubin架构+NVL144超节点以PD分离方式部署推理服务将成为最佳实践。

不仅NVIDIA，多家异构计算硬件厂商均在规划更单大规模GPU/TPU高带宽域的超节点形态基础设施。

总结与展望

相较于2024年，可能相同参数量的模型，能力已经有了超过十倍的差异，这些进步是模型、系统、数据等多方面综合进步的结果。2025年，工程和系统方面的工作整体方向是更精细化的设计和优化，而非一味的追求“超级硬件”和“超级集群”了。

2026年，系统的优化工作将会更加精细化，与进一步探索基于低成本硬件的极致性价比；而工程重点发展方向应该在AI Agent Infra，因为所有模型、工程相关工作的进步全部是服务于AI应用真正的落地，“贾维斯”何时会来到我们身边，我想这应该不远了。

参考

1. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. https://arxiv.org/pdf/2401.09670

2. DOPD: A Dynamic PD-Disaggregation Architecture for Maximizing Goodput in LLM Inference Serving. https://arxiv.org/pdf/2511.20982

3. Step-3 is Large yet Affordable: Model-system Co-design for Cost-effective Decoding.https://arxiv.org/pdf/2507.19427

4. DeepEP.GitHub - deepseek-ai/DeepEP: DeepEP: an efficient expert-parallel communication library

5. TileLang: A Composable Tiled Programming Model for AI Systems.https://arxiv.org/pdf/2504.17577

6. SeamlessFlow: A Trainer–Agent Isolation RL Framework Achieving Bubble-Free Pipelines via Tag Scheduling. https://arxiv.org/pdf/2508.11553

7. NVIDIA Rubin CPX Accelerates Inference Performance and Efficiency for 1M+ Token Context WorkloadsNVIDIA Rubin CPX.NVIDIA Rubin CPX Accelerates Inference Performance and Efficiency for 1M+ Token Context Workloads | NVIDIA Technical Blog





阅读 1.9万

留言

写留言

还没有留言，来写首评